

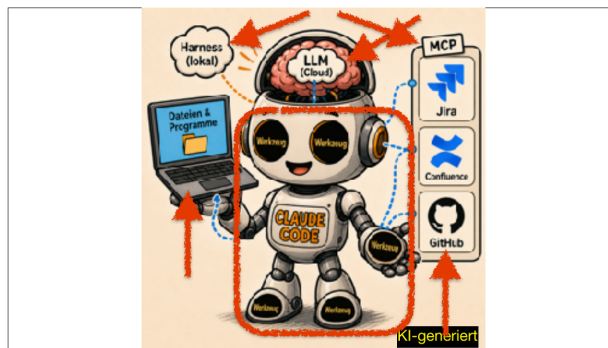
SCHNELLER ENTWICKELN MIT CODING AGENTS

David Kreutzer & Karsten Silz 23. Juni 2026

Was ist ein Coding Agent?

Agent

- KI, die Werkzeuge nutzt
- Agency
 - Agent kommt von "Agency"
 - Bedeutet Handlungsfähigkeit - eigene Handlungen und deren Konsequenzen bewusst steuern und beeinflussen



- Zwei Teile des Coding Agents: LLM & Harness
- Lokale Werkzeuge: Dateien, Programme (vor allem im Terminal), Web-Suche
- Werkzeuge im Netz: Model Context Protocol (MCP)

LLM

- Text nach Wahrscheinlichkeit
 - Autocomplete Engine für Texterzeugung
- Tokens
 - 1 Token = ungefähr 1 Wort
- Context Window
 - Kurzzeitgedächtnis der LLM
 - Alle Werkzeuge, Skills, Agenten, Dateien, Fragen, Antworten, Eingaben, Ausgaben...
- Komprimierung
 - Context Window voll: Komprimierung von 170-200k Tokens auf 10k
 - Wie Buch mit 300 Seiten ("Effi Briest") auf 10 Seiten zusammenfassen
 - Möglichst vermeiden, weil die meisten Informationen dann verloren gehen
 - Besonders schlimm: Mehrere Context-Komprimierungen hintereinander

Harness

- UI
 - UI: Terminal oder Graphisch
- Tools & Features
 - Lokale Tools
 - Dateien lesen, schreiben, durchsuchen
 - Websuche
 - Programme starten
 - Skills & Subagents sind Features
- Planung
 - Harness entscheidet oft selbständig, wann sie einen Plan präsentiert oder den Nutzer fragt
- CLAUDE.md
- AGENT.md
 - CLAUDE.md nur bei Claude, die meisten anderen Harnesses nennen es AGENT.md
 - Datei automatisch immer im Context Window
 - Wichtige Informationen hier ablegen – oder zu wichtigen Dateien verlinken

Coding Agent

- KI-EntwicklerTeam
 - PO, Analyst, Programmierer, Tester, Admin, ...
- Macht Fehler!

Anthropic	OpenAI	Google	Microsoft
Haiku	GPT Spark	Gemini Flash	MAI-Code-1-Flash
Sonnet	GPT	Gemini	MAI-Thinking-1
Opus	GPT Pro	Gemini Pro	
Fable		Antigravity CLI	GitHub Copilot CLI
Claude Code	Codex CLI	Antigravity	GitHub Copilot App
Claude Desktop	Codex		

- Claude Code, Codex CLI: Terminal-Tools
- Claude Desktop: Graphisches Tool mit Cowork - "Claude Code für Nicht-Entwickler" - und eingeschränktem Claude Code
- Codex: Graphisches Tool
- Anthropic Fable: <https://www.anthropic.com/news/claude-fable-5-mythos-5>
- Antigravity CLI, GitHub Copilot CLI: Terminal Tools
- Antigravity, GitHub Copilot App: Graphische Tools
- Migration von Gemini CLI zu Antigravity CLI: <https://an-important-update-transitioning-gemini-cli-to-antigravity-cli/>
- Neue Microsoft-LLMs: <https://microsoft.ai/news/building-a-hillclimbing-machine-launching-seven-new-mai-models/>

- GitHub Copilot Updates: <https://github.blog/news-insights/product-news/github-copilot-app-the-agent-native-desktop-experience/>
- Claude, Codex & Antigravity: Gut genug
 - Benchmark: SWE-bench - <https://www.swebench.com/>
 - Ergebnisse aktueller LLMs sind dort nicht mehr - werden nur noch vom Hersteller veröffentlicht
 - Aktuelle LLMs sind alle über 75%

Hat die KI recht?

- Lösung bekannt: 👍
 - Lösung bekannt: Meist bei Features, manchmal bei Bugs
 - Dann wissen wir, ob die KI recht hat
- Lösung unbekannt: 🙄
 - Lösung unbekannt: Meist bei Bugs oder Migrationen, manchmal bei Features
 - Dann wissen wir nicht, ob die KI recht hat
- KI prüft KI
 - Gleiche LLM oder unterschiedliche
- Mensch prüft KI
 - Mensch prüft: z.B. Code Review oder PR review
- "AI makes people in the know more powerful"
 - Aus einem Interview mit Benedict Evans - <https://www.ben-evans.com>
 - Nur wenn ich den Code lesen kann, weiß ich, ob er richtig ist

Wie behandeln?

- Nicht-deterministisches, vergessliches Junior-Team – schreiben & testen lassen
- Team
 - PO/PM, Analyst, Entwickler, Tester, Admin, Security Specialist, ...
 - Ich bin KI-Teamleiter, nicht Programmier
 - Als Chef bin ich schuld, wenn's schiefgeht
 - Meine Produktivität hängt davon ab, wie viele Agenten ich gleichzeitig wie gut steuern kann
 - Der Erfinder von Claude Code hat immer 5 Claude-Code-Instanzen parallel laufen
- Junior
 - Junior-Entwickler brauchen Onboarding
 - Werkzeuge & Kontext
 - Pläne prüfen
 - "Erfolg" lehren
 - Werkzeuge & Context
 - Zugriff auf meine Werkzeuge (Tickets, Wiki, Repos, ...)
 - Domäne: Was ist das Fachgebiet?
 - Spezifikation: Was macht Anwendung wie?
 - Wie bauen, testen, deployen und überwachen?

- Was immer tun - und was nie? Pläne prüfen
 - Plan für kleinere Aufgaben
 - Spezifikation + Plan für größere Aufgaben "Erfolg" lehren
 - Wie testen – und was?
 - Wie Produktion überwachen?
- Nicht-deterministisch
 - Software ist deterministisch: Gleiche Eingaben = gleiche Ausgaben
 - LLMs sind nicht-deterministisch: Gleiche Eingabe = manchmal andere Ausgabe
- Vergesslich
 - Coding Agents lernen nicht von ihrer Arbeit und erinnern sich nicht - sehen Code beim Start zum ersten Mal
 - Claude Code Subagents erinnern sich manchmal - MEMORY.md und *-pattern.md
- schreiben lassen
 - Wer soll all die Spezifikationen, Pläne und Dokumente schreiben?
 - Coding Agent schreibt, Mensch prüft
- testen lassen
 - Coding Agents führen die Back-End- und Front-End-Tests
 - Damit wissen sie, wann sie fertig sind
 - Front-End-Tests gern mit Playwright MCP - Browser-Steuerung (auch headless)
- KI nervt?
- Junior-Test!
 - Hätte ein Junior-Entwickler das am ersten Tag allein mit diesen Unterlagen geschafft?

Ideale Aufgaben

- Nicht jeder mag Coding Agents
- Bugs & Migrationen
 - Zum Beispiel Spring Boot 3 zu Spring Boot 4

Zusammenfassung

- LLM & Harness, die Werkzeuge nutzt
- Nicht-deterministisches, vergessliches Junior-Team – schreiben & testen lassen

Skills und Subagents

Subagents

- KI-Spezialisten
 - Analyst, Front-End-Entwickler, Back-End-Entwickler, Tester, Admin, ..
 - Teil der Harness nicht der LLM
- Gleiche LLM
 - Eher Personas
 - Konfiguration direkt im Subagent

ba-writer	admin
db-coder	ui-designer
be-coder	be-test-coder
fe-coder	fe-test-coder

- ba: Business Analyst
- be: Back End
- fe: Front End
- Schneller besseren Code
 1. Eigenes Context Window
 2. Laufen parallel
 3. Regeln anpassbar
 - Mehr Regeln als in CLAUDE.md
 - Höhere Wahrscheinlichkeit, dass Regeln beachtet werde
 4. Lernen
 5. Reviewer Subagents

ba-writer	admin	ba-reviewer	
db-coder	ui-designer	db-reviewer	ui-reviewer
be-coder	be-test-coder	be-reviewer	be-test-reviewer
fe-coder	be-test-coder	fe-reviewer	fe-test-reviewer

- Reviewer Subagents haben eigenes Context Window und sehen Code zum ersten Mal
- Nichtdeterministisch: Wählen vielleicht anderen Ansatz als Coder-Subagents
- Im Ergebnis finden sie mehr Fehler als Coding Agent allein
- Erstellt in Claude
- Direkt editieren
 - /agents, dann "New Agent"
- Nichtdeterministisch?
- 👍
 - LLMs sind so
 - Reviewer-Agents habe andere, frische Perspektive

Skills

- Automatisierte Workflows
- /plan-and-do
 - Vom Ticket/freier Aufgabe zum PR

- Teil der Harness, nicht der LLM
- Branch => (PRD =>) Plan => Code => Tests => Review => PR
 - PRD: Product Requirements Document
- Prompts + Skripts/ Programme
- Global oder Projekt
 - Prompts: nicht-deterministisch
 - Skripts/Programme: deterministisch
 - Global: in ~/.claude/skills
 - Projekt: in .claude/skills
- CLAUDE.md
 - Zum Beispiel: Test-Kommando und Liste der Subagents
- /plan-and-do Write a skill that does...
 - Warum /plan-and-do? Nutzt Subagenten, hat Reviews, macht Git Branch & PR
- /plan-and-do Change skill so...
 - Ich editiere Skills nicht direkt
- Nichtdeterministisch?
 - LLMs sind so
- 👍
 - Skills sind ein gutes Gerüst für Nondeterminismus
- 👎
 - Nur selten machen sie Mist: /plan-and-do ignoriert dann Prompts und läuft einfach durch

Zusammenfassung

- Skills automatisieren Workflows
 - Vom Ticket/freier Aufgabe zum PR
 - Teil der Harness, nicht der LLM
- Subagents schreiben schneller besseren Code
 - Vom Ticket/freier Aufgabe zum PR
 - Teil der Harness, nicht der LLM